

Proposal: Designing a Declarative Memory Layout Language for C

Kae Chinchilla 1526925

2025-03-05

1 Introduction

Precise control over memory layout is a core requirement in systems programming, where performance [9], binary compatibility, and hardware interaction often depends on the exact physical arrangement of data in memory [20]. The C programming language, long dominant in this domain [2, 3], offers minimal abstractions over layout, relying on low-level mechanisms such as bit-fields, unions, [33] and compiler-specific pragmas [12]. While flexible, these techniques are error-prone, difficult to verify, and notoriously fragile across compilers or platforms [29]. Layout bugs in C can cause subtle performance issues, memory corruption, or security vulnerabilities —particularly in contexts such as device drivers, embedded systems, and protocol implementations [16, 21].

Several approaches have emerged to address layout and memory related challenges and safety, including serialization frameworks (e.g., FlatBuffers [14], Cap’n Proto [25]), safe systems languages (e.g., Rust [31, 17]), and formally verified tools such as Cogent and Dargent [6]. While these tools offer valuable insights, most are either embedded in their restricted ecosystems or lack general applicability to existing C codebases. In particular, there remains a lack of dedicated tooling that allows systems programmers to write layout declarations declaratively, check them for correctness against C types, and do so without abandoning the flexibility and portability of C.

Cedar is a research prototype that attempts to fill this gap. It introduces a layout description language designed to be used alongside C, enabling declarative layout specifications while supporting semantic checking for type

compatibility. However, the current implementation is limited, making it difficult to apply in real-world use-cases.

This project aims to implement, design, and evaluate Cedar by refining its language design, strengthening its implementation, and applying it to representative systems programming scenarios. Through a structured methodology involving critical analysis, language extension, and case-study based evaluation, the project seeks to assess whether Cedar can realistically improve layout safety, expressiveness, and maintainability in C. In doing so, it contributes to the broader effort of making low-level programming more reliable without sacrificing control.

2 Background

In systems programming, precise control over data layout is essential for achieving high performance, interoperability, and correctness [9]. Languages such as C offer minimal abstraction over memory, granting programmers the flexibility to write highly efficient code, but also placing a significant burden on them to manually define binary layouts through constructs such as bit-fields, unions, and alignment directives. While this low-level control is powerful, it is notoriously error-prone [21, 29], particularly in contexts where memory correctness is critical, such as device drivers, embedded systems, and network protocols [34].

Traditionally, developers have addressed these challenges by writing manual marshalling and unmarshalling code [22], or by using unsafe macros to convert between data structures and low-level binary representations. These techniques are difficult to maintain, costly in performance [34], and extremely challenging to formally verify [4]. As a result, layout-related bugs often lead to serious vulnerabilities, system crashes, or subtle performance degradations [29]. A fundamental contributor to this problem is the lack of clear abstraction between what a data structure represents semantically and how it is laid out in memory, complicating both human reasoning and automated analysis.

Real-world examples highlight the seriousness of these issues. In the development of device drivers, for instance, hardware registers often impose strict alignment and bit-level layout constraints that must be matched precisely [3]. Misinterpreting these layouts can lead to misconfiguration of hardware devices, data corruption or complete system failure [29]. Similarly, in

network protocols stacks must adhere to standardized packet formats, where incorrect struct definitions can result in vulnerabilities exploitable by malformed input [5]. These risks underline the critical importance of providing developers with better abstractions for defining and verifying data layouts.

Recent research has explored more principled approaches to layout description. One notable example is **Dargent**, a data layout language integrated into the **Cogent** ecosystem [6]. Dargent enables programmers to declaratively specify memory layouts for algebraic data types and supports automatic generation of layout-aware C code with formal correctness proofs, via integration with the Isabelle/HOL theorem prover. However, Dargent is tightly coupled to Cogent, a restricted functional language that, while powerful for verification purposes, may be difficult to adopt as it requires programmers to work in a completely different language.

To address this gap, the Cedar project proposes a data description language designed to operate directly in the C ecosystem. Cedar enables programmers to define the memory layout of C structs through a high-level declarative syntax, while preserving compatibility with C’s expressive and performance-oriented features. Implemented as a preliminary prototype compiler in Haskell, Cedar currently supports basic layout declarations. However, its design remains at an early stage and lacks the robustness and expressiveness required for practical adoption in systems programming.

This project builds upon the foundational work of Cedar. Its goal is to investigate how the language and toolchain can be extended to support a broader range of C layout idioms, improve tooling robustness, and better serve the needs of systems programmers. In doing so, it aims to explore the design principles necessary to develop a sound and practical data description language for C, balancing flexibility, safety, and usability.

3 Literature Review

A robust understanding of prior work is essential to motivate and inform this project, which investigates how to make declarative memory layout descriptions in C more expressive, practical and safe for systems programming. This section surveys the current state of the art in memory layout control, beginning with the challenges posed by low-level programming in C and the risks of manual layout specification. It then examines existing techniques and tools used to mitigate these challenges, including runtime sanitizers and

marshalling libraries. Further, the review explores modern systems programming languages and DSLs, such as Rust, Cap’n Proto, and FlatBuffers, that aim to improve layout safety and correctness. Finally, the literature review focuses on verified layout description approaches, particularly the Dargent system embedded in Cogent, and Cedar—the subject of this project—which seems to adapt these ideas to the full C language.

Together, these works outline both the need for and the limitations of current approaches, and provide a foundation for the investigation carried out in this research.

3.1 Systems Programming and Data Layout Challenges

Systems programming is a field of computer science that focuses on the design and implementation of low-level software that interacts directly with hardware and system resources. This includes operating systems, device drivers, embedded systems, and other performance-critical applications. [21] [5] Systems programming is characterized by the need for high performance, fine-grained control over hardware resources, and efficient use of limited computational resources. These demands require a deep understanding of computer architecture, memory management, and low-level programming constructs [33].

Languages such as C and C++ have traditionally been the primary tools for systems programming, due to their minimal abstractions over hardware and their ability to produce highly efficient machine code. [3] However, this low-level access comes at the cost of increased complexity and potential for serious errors [21]. In particular, the task of managing data layout—determining how structured data is represented and accessed in memory—poses significant risks [23]. Programmers must carefully control how structures are packed, how fields are aligned, and how memory is accessed, often across different platforms with varying architectural requirements.

Manual control over data layout is critical in many systems applications. For example, device drivers must interact with hardware registers that demand exact field alignments and offsets [30]; network stacks must correctly parse protocol [5] headers with specific bit-level representations; and file systems must serialize and deserialize data structures according to on-disk formats. [28] Errors in data layout handling can lead to severe consequences, including memory corruption, security vulnerabilities, and unpredictable behavior [29, 34, 16].

Despite the importance of correct data layout, C provides little direct support for verifying or enforcing these properties. While some compiler-specific directives exist (such as `#pragma pack` and compiler-specific attributes for struct alignment) [11], their use is often fragile, non-portable, and difficult to reason about formally. Programmers must manually ensure the correctness of layouts, which increases maintenance costs and the likelihood of bugs.

These challenges are not merely theoretical.

Numerous real-world vulnerabilities—such as buffer overflows, misaligned memory accesses, and serialization bugs—can be traced back to subtle layout issues. Consequently, there has been significant interest in developing tools, languages, and frameworks that help programmers specify and reason about data layouts more safely and declaratively, without sacrificing performance.

3.2 Manual Approaches and Current Tools

In response to the challenges of data layout management in C, developers have traditionally relied on a combination of manual techniques and runtime debugging tools to mitigate layout-related errors. These approaches while effective to a degree, remain fundamentally limited in their ability to ensure correctness, safety, and long-term maintainability.

At the language level, C provides several mechanisms to manually control memory layout. Programmers can use `struct` field ordering, bit-fields, union types, and compiler-specific derivatives such as `#pragma pack`, `__attribute__((packed))`, or alignment hints to influence how data is laid out in memory [7]. These features are widely used in systems programming, particularly when interacting with hardware or network protocols that require binary compatibility with an external specification.

Despite their widespread use, these techniques are notoriously fragile. Their behavior can vary across compilers and platforms [11], and small changes in struct definitions may silently alter the resulting layout due to implicit padding and alignment adjustments. As a result, layout correctness often depends on careful inspection and trial-and-error, rather than principled guarantees.

To aid in the detection of layout-related bugs at runtime, developers commonly employ dynamic analysis tools such as **Valgrind** [27] and **AddressSanitizer (ASan)** [26]. Valgrind performs memory instrumentation to detect invalid memory accesses, buffer overflows, and uninitialized memory reads. AddressSanitizer, a compiler-based instrumentation tool, can detect

a range of memory errors including out-of-bounds accesses and use-after-free bugs. Similarly, **Undefined Behavior Sanitizer (UBSan)** [35] focuses on catching integer overflows, type punning violations, and alignment issues. These tools are invaluable for debugging complex memory errors and are widely used in systems development and testing pipelines.

While these tools are invaluable for finding certain classes of bugs, such tools are reactive rather than preventive. They identify errors only after they occur during runtime, meaning that test coverage gaps may leave bugs undetected. Moreover, they often impose substantial performance overhead and may not be suitable for deployment in embedded or real-time systems. Most critically, they provide no *no compile-time guarantees* that a memory layout adheres to its intended specification. This makes them inadequate as a solution to the broader problem of layout correctness, especially in safety- or security-critical environments.

In summary, while manual techniques and dynamic analysis of some tools analysis tools form an essential part of the system programmer’s toolbox, they do not eliminate the fundamental risks associated with low-level memory layout management. These shortcomings motivate the exploration of alternative approaches that aim to make layout specification more declarative, reliable, and amendable to formal reasoning.

3.3 Modern Safer Alternatives

In recent years, a number of modern systems programming languages and serialization frameworks have been developed in an effort to improve the safety and reliability of low-level programming, particularly in the context of memory layout and data representation. These tools aim to reduce the risk of memory corruption, undefined behavior, and layout mismatches that are prevalent in traditional C-based development.

One of the most prominent language-based alternatives is **Rust** [13], a systems programming language designed with memory safety as a central goal. Rust’s type system, borrow checker, and ownership model collectively present many classes of memory errors at compile time, including use-after-free bugs, dangling pointers, and data races. In terms of memory layout control, Rust offers the `#[repr(C)]` attribute, which instructs the compiler to lay out structs and enums in a manner compatible with C [24]. This enables Rust code to safely interface with C libraries and binary formats, provided that the layout assumptions are correct.

However, Rust’s approach to layout control is not without limitations. While `#[repr(C)]` preserves field order and basic layout compatibility, it does not prevent all sources of layout divergence. The language still permits implicit padding, alignment rules vary by platform, and there are no native mechanisms for specifying bit-level layout or enforcing alignment constraints beyond what the compiler already provides. Furthermore, complex or performance-critical layout manipulations in Rust often require the use of `unsafe` code, which bypasses the language’s safety guarantees. In such cases, developers are again responsible for manually ensuring correctness, returning to the same fragile patterns common in C [8].

In addition to language-level solutions, a number of serialization frameworks have been developed to simplify the task of representing structured data in binary form. Among the most widely adopted are **Protocol Buffers (Protobuf)** [18], **Cap’n Proto** [25], and **FlatBuffers** [14]. These tools allow developers to define data schemas using an external language or IDL (Interface Definition Language), from which code is generated to handle serialization, deserialization, and (to varying degrees) in-place memory access.

Cap’n Proto, in particular, emphasizes zero-copy access to serialized data by allowing direct memory access without requiring explicit marshalling or unmarshalling. **FlatBuffers** similarly supports in-place data parsing with minimal runtime overhead. These frameworks are widely used in networked applications, game engines, and interprocess communication systems, where both performance and portability are critical.

Despite their strengths, these tools are not general-purpose solutions for memory layout verification in systems programming. Their primary focus is on serialization and interoperability between languages and platforms, rather than enabling fine-grained layout control in low-level systems code. While they offer high performance and strong schema evolution guarantees, they operate in constrained environments where data is accessed through framework-generated APIs, not as arbitrary pointers or unions typical in C-based systems. As such, they are not well suited for tasks such as device driver development, interacting with memory-mapped hardware, or enforcing precise layout guarantees in systems-level components.

Moreover, these frameworks often impose restrictions on data-structures and memory access patterns, which can limit their utility in performance-critical or hardware-near contexts. Integration with legacy C codebases is also non-trivial, requiring translation layers or FFI binding that may intro-

duce overhead or compatibility issues.

Another emerging project of interest is **Carbon**, a language initiated by Google as a possible successor to C++ [32]. Carbon is intended to provide modern language features, safety guarantees, and improved tooling while maintaining strong interoperability with existing C++ codebases. Although still in an early stage of development, Carbon represents a broader movement toward rethinking systems programming foundations. However, it remains unclear whether Carbon will support fine-grained memory layout control or formal verification of layout correctness—features that are essential for the kinds of systems-level application Cedar targets. As such, while Carbon highlights growing awareness of the limitations of C and C++, it does not yet offer a concrete solution to the specific challenges this project addresses.

In summary, while tools like Rust, Cap’n Proto, FlatBuffers, and Carbon represent meaningful progress toward safer and more expressive memory manipulation, they do not fully address the problem of reliable, declarative memory layout specification in systems programming. None of these solutions offer the ability to define layouts with the precision required for low-level systems work while simultaneously ensuring structural correctness and usability across diverse C environments. This gap continues to motivate the development of domain-specific languages and toolchains—such as Cedar—that aim to offer greater control and safety without sacrificing low-level flexibility.

3.4 Verified Systems Programming: Cogent and Dargent

An alternative approach to managing layout correctness in systems programming is to shift the focus from error detection to *formal verification*. Rather than relying on runtime tools or manual inspection, formal methods aim to *prove* properties of programs, including memory safety, and layout correctness, with mathematical certainty. A notable system in this space is **Cogent**, a functional programming language designed to generate verifiable and efficient C code for low-level systems components. [6]

Cogent was developed with the goal of producing systems code that is both performant and formally verifiable. It achieves this by enforcing a total, pure, and strongly-typed programming model, from which it is possible to extract formal semantics. The Cogent compiler generates both C code and a

corresponding formal representation in Isabelle/HOL, a proof assistant used to verify properties of the generated code. This model enables refinement proofs showing that the low-level C code behaves exactly as specified by the high-level Cogent program.

Within the Cogent ecosystem, **Dargent** serves as a domain-specific language for describing data layouts/ Dargent allows developers to specify memory representation of algebraic data types with precise control over bit-level layouts, offsets and alignment. This is particularly valuable in scenarios such as device drivers or file systems, where data must conform to strict binary formats. Unlike traditional approaches that require hand-written marshalling logic or inline bit manipulation, Dargent integrates layout descriptions into the language itself and automates the generation of accessors that manipulate the raw memory safely.

A key feature of Dargent is its **separation of types and layouts**: a programmer defines the logical structure of a data type independently of its physical representation. Layouts are then associated with types explicitly, allowing layout specifications to evolve independently from program logic. This separation facilitates reuse, modularity, and composability—principles that are often missing in systems programming languages where layout is tightly coupled to struct definitions.

Dargent also performs static well-formedness checks on layout declarations to ensure memory safety. These checks validate the fields do not overlap, are correctly aligned, and match the semantics of the associated types. The Cogent compiler then generates C code that includes custom getter and setter functions for accessing the laid-out memory, along with machine-checked proofs that these functions correctly implement the layout semantics. As a result, Dargent provides a rare combination of **bit-level control**, **zero-copy memory access** and **formally verified correctness**.

Despite these advantages, Dargent and Cogent operate within a deliberately restricted domain. Cogent intentionally omits features such as recursion, mutable state, and unrestricted loops in order to preserve the ability to formally verify generated code/ While this makes Cogent an excellent choice for verified components in systems such as file systems or microkernels, it also limits its general applicability. Furthermore, Dargent’s utility is tightly bound to Cogent’s type system and verification infrastructure, making it less suitable for integration into broader, unverified C codebases.

These limitations highlight the need for tools that offer layout specification and correctness guarantees in a **less restrictive and more in-**

teroperable setting. In particular, there is an opportunity to adapt the ideas behind Dargent—declarative layouts, type-layout separation, and layout validation—to tools designed for general C programming. This is the motivation behind Cedar.

3.5 Cedar: Toward Practical Declarative Layouts for C

Cedar is an early-stage research project that explores how declarative layout specification techniques—such as those developed in Dargent—might be adapted to work in the C programming ecosystem. Unlike Dargent, which is tightly integrated into the Cogent language and its verification framework, Cedar is intended to work as an independent tool that extends C with high-level layout descriptions in a form that is both expressive and, potentially, verifiable.

Cedar proposes a syntax for defining memory layouts that is decoupled from type definitions in C. Programmers can describe layouts using constructs such as `layout`, `at`, `before`, and `after`, which are then associated with C `struct` types through a linking mechanism. This design borrows the ideas of type-layout separation from Dargent, allowing developers to describe physical data representations independent of logical structure. However, unlike Dargent, Cedar does not rely on a restricted or pure functional language. It is intended to work directly with C code, making it more accessible for real-world systems development.

The current prototype, implemented in Haskell, includes a semantic analysis phase, which operates on reduced representations of layouts and C types. These representations are obtained through a basic parser. Once reduced, Cedar applies a structural matching relation to check for compatibility between layouts and their associated C types. This allows the tool to detect mismatches in field order, size, or layout structure at an early stage. While it does not offer verification or compiler integration, this form of analysis provides strong syntax checks.

Cedar currently supports layouts for structures, arrays and tagged unions, reflecting common use cases in systems programming. However, the implementation remains incomplete in several important areas. It lacks support for low-level constructs such as untagged C unions, flexible array members, nested layout references, and overlapping bitfields. Diagnostic support is

minimal, error messages are not yet developed, and the toolchain does not include features like layout visualization, code generation or IDE integration. These limitations significantly restrict usability in practice.

An important design goal of Cedar is eventual compatibility with **CompCert**, a formally verified C compiler [15]. CompCert translates C code into assembly, while providing machine-checked proofs of semantic preservation, making it particularly attractive for use in safety- and security-critical systems. However, CompCert’s correctness guarantees assume that the source C program is well-formed and layout-safe. It does not reason about whether struct layouts corresponding to external binary formats or hardware specifications. As a result, layout mismatches—such as overlapping fields, incorrect offsets, or misaligned access—can still lead to undefined behavior or runtime errors, even when compiling with CompCert.

Cedar aims to fill this gap by statically analyzing working with layout declarations, guaranteeing they are structurally compatible with their associated types. In doing so, It provides an additional layer of safety and confidence when writing low-level code intended for verified compilation. Although integration with CompCert is not currently in the scope of the project, Cedar’s design positions it as a potential framework or pre-analysis tool that could generate layout-safe C code suitable for verified compilation.

In summary, Cedar represents a promising but incomplete approach to declarative specification in C. It borrows key insights from Dargent—particularly type-layout separation and structural layout matching—but repurposes them for a more general context. By complementing verified toolchains like CompCert with layout-level checking, Cedar has the potential to improve correctness, expressiveness, and maintainability in C-based systems development. This project seeks to explore and extend Cedar’s capabilities toward that goal.

4 Problem Statement and Objectives

Despite the critical role of data layout in systems programming, mainstream languages like C provide only low-level, error-prone mechanisms for controlling memory representation. Developers must rely on fragile constructs such as bitfields, unions, and compiler-specific pragmas to enforce binary compatibility, often leading to layout mismatches, security vulnerabilities, or undefined behavior. While tools like Dargent demonstrate the feasibility of

declarative, verifiable layout specifications, they are embedded within restricted language ecosystems and are not applicable to general-purpose C codebases.

Cedar proposes a promising alternative by introducing a standalone layout description language for C, but its current implementation is at a prototype stage. It lacks full support for common layout idioms, has limited diagnostic feedback and does not yet offer a robust or user-friendly workflow for systems programmers. In addition, it remains unclear how expressive or usable Cedar is in practical low-level programming scenarios, or how its design could evolve to better serve real-world needs.

There is therefore a need to evaluate, refine and extend Cedar’s design and implementation to make it a more robust and effective tool for declarative memory specification in C.

This project aims to explore the design space of declarative layout languages for C by extending and evaluating Cedar. The specific objectives are:

- **Identify key limitations** in Cedar’s current syntax, semantics and toolchain through analysis of C layout scenarios.
- **Design and implement extensions** to improve its expressiveness, including support for additional C constructs.
- **Improve semantic analysis and diagnostics**, enabling more helpful and precise error reporting when layout mismatches occur.
- **Develop a suite of presentative case studies** using C data structures to evaluate the practicality and expressiveness of Cedar’s layout language.
- **Assess the potential for integration** with verified toolchains outlining necessary steps and future directions for formal layout verification.

By addressing these objectives, this project aims to establish whether a tool like Cedar can realistically improve layout readability and maintainability in low-level systems code, without imposing the restrictions of fully verified languages.

5 Methodology

This section outlines the methodology adopted to address the research objectives identified in Section 4. The project lies at the intersection of programming language design, tool development, and low-level systems engineering. The core of our project is to aim is to implement and evaluate Cedar—a declarative language for specifying the memory layouts of C data structures. Our work will provide an implementation of the language an early-stage prototype [1]. Then we plan to validate Cedar in terms of correctness and practical utility by applying it to testing on several case studies.

This goal requires a combination of literature-informed design analysis, critical evaluation of Cedar’s current implementation, targeted language and tooling enhancements, and empirical evaluation through case studies. The methodology is structured into distinct but interrelated phases, each contributing to the iterative refinement and validation of the proposed extensions.

5.1 Research Methodology Overview

This project is framed as a **design-oriented systems research investigation**, with the primary objective of advancing the design and implementation of a domain-specific language (DSL) for declarative memory layout specification in C. The methodology reflect a **constructive research paradigm**, in which a software artifact (Cedar) is incrementally developed, extended and evaluated to explore its capacity to address an identified gap in existing systems programming practices. Rather than seeking to establish universal truths through empirical generalization, the project aims to generate insight through **iterative design, critical reflection, and focused case-based evaluation**.

The research proceeds through a sequence of methodologically distinct but conceptually aligned phases. The first is an analysis of the **design**, grounded in literature on verified systems programming, layout correctness, and DSL design. This stage establishes theoretical foundations and identifies prior approaches (e.g., Dargent, Rust, Cap’n Proto) that inform the Cedar’s conceptual model. This phase also involves close reading of the Cedar prototype and supporting documentation to extract the underlying assumptions and design goals.

The second phase focuses on **design evaluation**, involving a detailed as-

assessment of Cedar’s current syntax, semantics and tooling capabilities. This includes a critical analysis of what layout constructs it supports, what guarantees (if any) it provides about layout-type compatibility and how usable and expressive the language is in its current form. This stage informs the third phase: the **development of targeted extensions** to Cedar, aimed at addressing specific designing limitations and enabling new functionality or use cases.

Following this, the project enters an **empirical evaluation phase**, in which Cedar is applied to a suite of representative low-level programming tasks. These cases studies are conceptualized or directly drawn for real-world domains such as device drivers, embedded systems, or protocol parsers, where layout correctness is essential. The goal is not exhaustive benchmarking, but rather a focused demonstration of language expressive, usability, and limitations in practice.

Throughout the project, a strategy of **iterative refinement** is employed. Design changes are informed by insights gained from implementation experience and case study application, and are continuously reassessed for coherence with the project’s broader goals. This methodology mirrors established programming language research, which emphasizes co-evolution of language design, implementation and real-world usage.

This approach allows the project to remain grounded in practical needs, while retaining a principled approach and research-informed orientation. The resulting contributions are expected to take the form of a more robust Cedar implementation, a set of documented design improvements, and an evaluation of Cedar’s feasibility as a layout specification tool for systems programming in C.

5.2 Evaluating Prior Work to Inform Cedar’s Extensions

The first phase of the project involves a structured evaluation on prior work that informs the extension and refinement of Cedar. This evaluation is guided by the central research question: *How can we design a data layout language that is both practical for systems programming, and sound in its treatment of C memory models?* The focus is on identifying foundational ideas, successful patterns, and known limitations from related systems languages that bear similar goals.

The most directly influential system is **Dargent**, which demonstrates the viability of separating data structure semantics from memory layout at a language level. Dargent’s design philosophy—especially its separation of types and layouts, static layout validation, and automated getter/setter generation—provides a compelling model for Cedar. However, Dargent’s reliance on Cogent’s restricted language features and verification infrastructure limits its general applicability, making it more of a conceptual reference than a direct implementation template.

The evaluation also draws on **Rust**, whose combination of memory safety and layout control via `#repr(C)` annotations provides an example of how modern languages integrate safety with low-level control. While Rust introduces key ideas such as alignment guarantees and explicit unsafe boundaries, it still relies on manual reasoning in complex cases, and lacks a dedicated DSL for layouts. These limitations point to the continued relevance of external layout specification tools like Cedar.

From the domain of data serialization, tools such as **Cap’n Proto** and **FlatBuffers** offer insights into schema-based layout definition and zero-copy access models. These frameworks demonstrate that layout descriptions can be declaratively compiled into efficient runtime access code. However, their focus on interoperability and high-level abstractions renders them unsuitable for low-level systems work, where precise control and compatibility with compiler semantics are paramount.

In terms of formal toolchains, **CompCert** provides the verification backend that Cedar ultimately targets. CompCert guarantees semantic preservation of C programs during compilation, but assumes that the input programs are layout-correct. It does not verify, for instance, whether struct fields are properly aligned to match external hardware specification. This leaves a critical gap at the interface between high-level C code and low-level binary expectations—a gap that Cedar aims to address through explicit layout validation.

Finally, the existing Cedar prototype itself serves as a key input to this evaluation. Understanding its internal architecture, assumptions, and limitations will inform how the project approaches extension and refinement. This includes analysis of the syntax it offers, the semantics it defines (e.g., layout-type matching), and the capabilities of its semantic analysis infrastructure. By critically evaluating the gaps between Cedar’s intended goals and its current form, the project will develop a grounded understanding of what aspects require redesign or augmentation.

This evaluative process establishes the theoretical and practical background necessary for the rest of the project. It guides the identification of missing language features, informs the selection of case studies, and motivates the direction and future implementation work. In doing so, it ensures that subsequent phases are both responsive to real-world systems programming challenges and informed by relevant prior research.

5.3 Language and Tooling Evaluation

Before extending Cedar, it is essential to understand the current capabilities and limitations of its language design and tool support. This phase of the project involves a systematic evaluation of the existing prototype in terms of syntax, semantics, supported features, and developer-facing tooling. The goal is to identify pain points, inconsistencies, and gaps that hinder Cedar’s practical usability or its ability to scale to the real-world programming tasks.

5.3.1 Syntax and Semantics

Cedar introduces a minimal declarative syntax for describing memory layouts using constructs such as `layout`, `at`, `before`, `after`, and `union`. These allow users to define bit- and byte-level layouts independently of their C type definitions. While this syntactic separation reflects a core strength of the language, preliminary analysis shows that the expressiveness of the current syntax is limited. For instance, there is no support for layout reuse (e.g., parameterized or nested) layouts), custom alignments, or layout macros, all of which could significantly improve modularity and readability.

Additionally, while Cedar supports tagged unions, the lack of support for certain idiomatic C constructs—such as flexible array members, anonymous structs/unions, and overlapping fields—restricts its applicability. These omissions can be especially problematic in operating system development, where such constructs are common. Addressing these gaps will be a central part of the design and implementation phases of the project.

5.3.2 Semantic and Type-Layout Matching

Cedar includes a semantic analysis stage that checks whether layout declarations are structurally compatible with associated C types. This is performed via a matching relation between reduced layout forms (LR), and reduced

C types (CR). While this relation captures many useful constraints—such as field order, bit width, and alignment—it remains limited in scope. It does not currently model more advanced C semantics such as padding rules, platform-specific ABI alignment, or complex nesting scenarios involving unions and structs. Moreover, layout matching failures often result in minimal or opaque error messages, reducing the utility of this phase for end users.

As part of this evaluation, the project will systematically test the limits of Cedar’s layout-type compatibility checking, both by reviewing the implementation of the semantics and by constructing case studies based on real-world C systems datatypes, including ones with headers and memory maps.

5.3.3 Implementation Quality and Tooling Gaps

Cedar’s implementation, written in Haskell, is modular and readable but remains at a prototype level. It lacks a dedicated parser integration with standard C toolchains. There is no error recovery system, layout visualization, or editor support, and the tool must be run manually on a per-file basis with limited feedback. Logging and diagnostics are minimal, making it difficult for users to debug mismatches between layout and C types.

There is also no test suite or regression framework, making it difficult to ensure semantic consistency or validate new features. This absence poses a significant risk to extensibility and maintainability, which will be addressed during the implementation phase of the project.

5.3.4 Summary

This evaluation phase will result in a detailed inventory of Cedar’s current capabilities and limitations, guiding the prioritization of future implementation tasks. It will also provide the foundation for later case studies, ensuring that language extensions are aligned with practical layout needs and are grounded on actual usage scenarios rather than hypothetical examples.

5.4 Design and Implementation Plan

This phase of the project involves extending the existing Cedar prototype to improve its expressiveness, correctness, and developer usability. The focus is on evolving Cedar from an experimental prototype into a more robust

research tool capable of supporting practical layout specification of datatypes used in real-world systems implemented in the C language.

The implementation work will build directly on the existing Haskell code-base, which serves as the foundation for Cedar’s semantic analysis. All changes will prioritize **modularity, maintainability, and clarity**, to ensure that future contributors can reason about and extend the tool with minimal overhead. The planned implementation tasks are informed by the evaluation in the previous phase, as well by ongoing case study development.

5.4.1 Designing and Implementing Language Features

A key implementation goal is to broaden the expressiveness of Cedar’s layout language. Several concrete extensions are planned, including:

- **Flexible array member support**, which is critical for struct compatibility with variable-length protocols
- **Nested layout references**, to allow layout composition and reuse
- **Improved union handling**, including tagged unions where safe,
- And the ability to express **alignment constraints** and **bitfield overlap**, which appear frequently in low-level hardware structures.

These extensions will be prioritized based on complexity, case study relevance, and risk. Where necessary, small, focused experiments will be conducted to determine implementation feasibility before fully integrating them into the language.

5.4.2 Improving Semantic Analysis and Diagnostics

Cedar’s current semantic matching system will be extended to support more precise and informative checks. This includes:

- Verifying field sizes and offsets with respect to the C standard’s alignment and padding rules.
- Improving error messages for layout-type mismatched, including reporting field-level conflicts.
- Providing more meaningful diagnostics for partially matched or ambiguous layouts.

Where possible, layout-type matching will be made more **transparent**, potentially via optional debug output or layout traces.

5.4.3 Testing and Validation Infrastructure

A lightweight **test suite** will be developed to validate semantic behavior across supported features. The suite will include:

- Unit tests for the matching relation and layout constructs
- Regression tests to ensure backward compatibility
- And adversarial examples drawn from real-world C libraries or hardware headers.

This test infrastructure will enable faster iteration, safer refactoring, and better confidence in semantic correctness.

5.4.4 Possible Extension: Tooling Improvements

If time permits, auxiliary tooling improvements may be explored, including:

- A **command-line interface (CLI)** for easier layout-type checking across projects,
- **Basic pretty-printing** or visualization of resolved layouts

These additions are not core to the research contribution, but may enhance the tool’s usability and adoption potential.

5.5 Case Studies and Evaluation

To evaluate the effectiveness of the extended Cedar language, the project will employ a series of structured case studies based on real-world systems programming tasks. These case studies are designed to test Cedar’s expressiveness, usability, and correctness when applied to representative layout scenarios encountered in embedded systems, drivers, and operating system components.

The case studies are selected to reflect the kinds of challenges that low-level C programmers routinely face when working with binary protocols, device drivers, and compact data formats. These scenarios typically involve

strict constraints on field alignment, bit-level packing, padding behavior, and platform-specific layout conventions. By applying Cedar to such examples, the project aims to assess both its practical capabilities and limitations.

We plan to consider three case studies were are used to evaluate Dargent [6]. This allows for a direct comparison between Cedar and its closest conceptual predecessor. Each case study reflects a common layout challenge in systems programming, and has been selected to test different aspects of Cedar’s design.

- **Power Control Structure**

A C struct representing a control block in a hardware power management unit. This structure typically includes fields for voltage targets, bit-level enable flags, and status registers packed into bytes.

- **CAN Driver Bitfield** A layout describing a message header in a CAN Controlled Area Network driver, often used in automotive and embedded systems.

- **Time Device Register Map**

A representation of a hardware timer’s memory-mapped control registers.

Additional case studies may be introduced depending on progress and time constraints, particularly targeting complex or non-standard layout idioms drawn from open-source C projects or Linux kernel headers.

5.5.1 Evaluation Criteria

Each case study will be assessed in the following criteria

- **Expressiveness:** Can the Cedar layout language describe the required layout accurately and concisely?
- **Correctness:** Does the semantic analysis correctly detect matches or mismatches between layout and type declarations?
- **Error Reporting:** Are diagnostic messages informative and helpful when mismatches or invalid constructs are encountered?
- **Usability:** Is the layout specification process intuitive and readable?

Where appropriate, manual comparisons will be made against traditional C representations, to illustrate Cedar’s advantages or trade-offs in terms of clarity, reliability and safety.

5.6 Iterative Refinement

Given the exploratory nature of language tooling design, this project will adopt an **iterative and feedback-driven development model**. Rather than attempting to define all language features or semantic rules up front, design decisions will evolve in response to insights gained from implementation, testing, and case study evaluation.

Each stage of the project—from language design and semantic analysis to case study application—may uncover limitations, ambiguities or opportunities for improvement. These findings will feed back into the design process through short reflection cycles, where the language syntax, semantic rules or tooling behavior may be refined to better accommodate real-world use cases or improve usability

For example, if a particular layout construct proves awkward to express or repeatedly triggers misleading diagnostics, its syntax or interpretation may be revised. If a case study exposes a semantic gap between Cedar and the C type system, the layout type matching algorithm may be extended or clarified. These refinements will be documented and justified, with particular attention to maintaining internal consistency and alignment with the project’s goals.

This iterative process mirrors established practices in programming language research, where prototype implementations are frequently refined in tandem with theoretical insights and practical experimentation [19, 10]. The final report will include a discussion of major design trade-offs encountered during this process, including reflection on usability, expressiveness, verification potential, and compatibility with C semantics.

By explicitly embracing feedback loops, the project aims to produce a more grounded and usable outcome— not only a theoretically sound language, but one that reflects the practical realities of low-level software development

5.7 Optional Extensions

While the primary focus of this project is on evaluating and extending Cedar’s language and analysis features, several optional extensions may be explored if time permits. These extensions align with the long-term vision for Cedar as a usable and verifiable layout specification tool, and may help lay the groundwork for future research or integration

5.7.1 Expanded Case Study Coverage

If the core case studies are completed ahead of schedule, additional test cases may be drawn from large open-source codebases such as the Linux kernel or device drivers. These examples often feature complex structures with non-standard layouts that would serve as a rigorous test for Cedar’s capabilities.

5.7.2 Formal Verification Integration

One long-term goal of Cedar is the eventual development of a verification backend. While a full backend is beyond the scope of this project, preliminary work may be done to assess how Cedar’s semantics might be formally encoded, and what information would need to be exported to verification pipelines. Even exploratory notes on this integration will be valuable for future projects.

5.7.3 Improved Developer Tooling

If time allows, basic tooling enhancements may be implemented to improve developer experience. Potential improvements include:

- A command-line interface (CLI) for batch checking layouts
- Syntax highlighting or editor plugins for Cedar layout files
- Structured output (e.g., JSON) for integration with other tools such as documentation generators.

6 Project Timeline

The project follows a semester-based timeline, with each phase aligned to specific weeks of the academic calendar. The schedule reflects both the it-

erative nature of the project and the phased structure of the methodology outlined previously. Early weeks were devoted to understanding the context, reviewing relevant literature, and planning the project design. The remaining weeks focus on implementation, evaluation, and iterative refinement.

- Weeks 1-2: Project commenced. Initial meetings with supervisor, familiarization with the Cedar prototype and relevant tooling
- Weeks 3-7: Literature review and project scoping. In-depth study of Dargent, CompCert, Rust, and related tools. Initial engagement with the Cedar codebase to understand its internal structure. Refinement of research objectives and drafting of early proposal sections.
- Week 8: Finalize the writing of the project proposal.
- Weeks 9-10: Begin development and refactoring
- Weeks 11-15: Design and implement key extensions.
- Weeks 16-17: Apply case studies to new implementation
- Weeks 18: Reflect on evaluation outcomes. Iterate on
- design if needed
- Week 19-21: Finalize language features
- Week 22-24 Buffer period for any unexpected delays.

7 Expected Outcomes and Impact

The project is expected to deliver both practical and conceptual outcomes that contribute to the usability and viability of Cedar as a tool for declarative layout specification in C. By extending the language’s expressiveness, improving its semantic analysis capabilities, and evaluating its design through representative case studies, the project will demonstrate the feasibility of using Cedar in realistic systems programming contexts.

The primary outcome will be an enhanced version of Cedar, featuring:

- A refined layout language with support for common systems-level constructs.

- Strengthened semantic analysis infrastructure capable of identifying layout-type mismatches with improved precision and diagnostics.
- A basic testing framework and documentation to support future development and experimentation.

In addition to these technical deliverables, the project will produce a detailed evaluation of Cedar’s strengths and limitations, grounded in real-world use cases. This evaluation will contribute insight into the trade-offs involved in designing a declarative layout DSL for low-level languages, and provide recommendations for future work, including paths toward formal verification or compiler integration.

Ultimately, this work aims to advance the broader effort of making systems programming safer and more maintainable. By offering a more expressive and analyzable alternative to ad hoc layout techniques, Cedar has the potential to reduce layout-related bugs, improve code clarity, and better align with low-level development practices with modern language design principles.

References

- [1] George Anderson. *Cedar, A Data Layout Description Language for C*. Nov. 18, 2024.
- [2] Henri E. Bal, Jennifer G. Steiner, and Andrew S. Tanenbaum. “Programming languages for distributed computing systems”. In: *ACM Comput. Surv.* 21.3 (Sept. 1989), pp. 261–322. ISSN: 0360-0300. DOI: 10.1145/72551.72552. URL: <https://doi.org/10.1145/72551.72552>.
- [3] Michael Barr. *Programming embedded systems in C and C++*. ” O’Reilly Media, Inc.”, 1999.
- [4] Joshua A Bockenek, Freek Verbeek¹, and Peter Lammich. “Formal Verification of Memory”. In: *Computer Safety, Reliability, and Security: 38th International Conference, SAFECOMP 2019, Turku, Finland, September 11–13, 2019, Proceedings*. Vol. 11698. Springer Nature. 2019, p. 35.
- [5] Guilin Chen, Mahmut Kandemir, and Mustafa Karakoy. “A constraint network based approach to memory layout optimization”. In: *Design, Automation and Test in Europe*. IEEE. 2005, pp. 1156–1161.
- [6] Zilin Chen et al. “Dargent: A silver bullet for verified data layout refinement”. In: *Proceedings of the ACM on Programming Languages* 7.POPL (2023), pp. 1369–1395.
- [7] David A Cock. “Bitfields and Tagged Unions in C: Verification through Automatic Generation.” In: *VERIFY 8* (2008), pp. 44–55.
- [8] Mohan Cui et al. “Is unsafe an Achilles’ Heel? A Comprehensive Study of Safety Requirements in Unsafe Rust Programming”. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 2024, pp. 1–13.
- [9] Nuno Faria, Rui Silva, and João L. Sobral. “Impact of Data Structure Layout on Performance”. In: *2013 21st Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. 2013, pp. 116–120. DOI: 10.1109/PDP.2013.24.
- [10] Chris Hankin, Hanne Riis Nielson, and Jens Palsberg. “Position statements on strategic directions for research on programming languages”. In: *ACM Sigplan Notices* 32.1 (1997), pp. 59–65.

- [11] Michael J Hohnka et al. “Evaluation of compiler-induced vulnerabilities”. In: *Journal of Aerospace Information Systems* 16.10 (2019), pp. 409–426.
- [12] Peder Jungck et al. “Pragmas and Other Key Compiler Directives”. In: *packetC Programming* (2011), pp. 233–235.
- [13] Steve Klabnik and Carol Nichols. *The Rust programming language*. No Starch Press, 2023.
- [14] Chaitanya Koparkar. “Efficient Data Representation Using FlatBuffers”. In: *XRDS: Crossroads, The ACM Magazine for Students* 29.2 (2023), pp. 50–51.
- [15] Xavier Leroy et al. “CompCert-a formally verified optimizing compiler”. In: *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*. 2016.
- [16] Xin Li et al. “A realistic evaluation of memory hardware errors and software system susceptibility”. In: *2010 USENIX Annual Technical Conference (USENIX ATC 10)*. 2010.
- [17] Per Lindgren et al. “Static and Dynamic Memory Protection using Type-Based Memory Layout in Rust”. In: *2023 IEEE 2nd Industrial Electronics Society Annual On-Line Conference (ONCON)*. IEEE. 2023, pp. 1–6.
- [18] Eduard Maltsev and Oleksandr Muliarevych. “Beyond JSON: Evaluating Serialization Formats for Space-Efficient Communication”. In: *Advances in Cyber-Physical Systems* 9.1 (2024), pp. 9–15.
- [19] Marjan Mernik and Viljem Žumer. “Incremental programming language development”. In: *Computer Languages, Systems & Structures* 31.1 (2005), pp. 1–16.
- [20] David Monniaux. “Verification of device drivers and intelligent controllers: a case study”. In: *Proceedings of the 7th ACM & IEEE International Conference on Embedded Software*. EMSOFT ’07. Salzburg, Austria: Association for Computing Machinery, 2007, pp. 30–36. ISBN: 9781595938251. DOI: 10.1145/1289927.1289937. URL: <https://doi.org/10.1145/1289927.1289937>.
- [21] Paul C van Oorschot. “Memory errors and memory safety: C as a case study”. In: *IEEE Security & Privacy* 21.2 (2023), pp. 70–76.

- [22] Francesca Pacini et al. “Performance analysis of data serialization formats in m2m wireless sensor networks”. In: *ewsn 2015* (2015), p. 7.
- [23] Neungsoo Park, Bo Hong, and Viktor K Prasanna. “Tiling, block data layout, and memory hierarchy performance”. In: *IEEE Transactions on Parallel and Distributed Systems* 14.7 (2003), pp. 640–654.
- [24] Graf von Perponcher-Sedlnitzki and Philipp Christian. “Integrating the future into the past: Approach to seamlessly integrate newly-developed Rust-components into an existing C++-system”. PhD thesis. Technische Hochschule Ingolstadt, 2024.
- [25] Deepti Raghavan et al. “Breakfast of champions: towards zero-copy serialization with NIC scatter-gather”. In: *Proceedings of the Workshop on Hot Topics in Operating Systems*. 2021, pp. 199–205.
- [26] Konstantin Serebryany et al. “{AddressSanitizer}: A fast address sanity checker”. In: *2012 USENIX annual technical conference (USENIX ATC 12)*. 2012, pp. 309–318.
- [27] Julian Seward and Nicholas Nethercote. “Using Valgrind to Detect Undefined Value Errors with Bit-Precision.” In: *USENIX Annual Technical Conference, General Track*. 2005, pp. 17–30.
- [28] Keith A Smith and Margo I Seltzer. “File system aging—increasing the relevance of file system benchmarks”. In: *Proceedings of the 1997 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*. 1997, pp. 203–213.
- [29] Vilas Sridharan et al. “Memory Errors in Modern Systems: The Good, The Bad, and The Ugly”. In: *SIGARCH Comput. Archit. News* 43.1 (Mar. 2015), pp. 297–310. ISSN: 0163-5964. DOI: 10.1145/2786763.2694348. URL: <https://doi.org/10.1145/2786763.2694348>.
- [30] Michael M Swift et al. “Nooks: An architecture for reliable device drivers”. In: *Proceedings of the 10th workshop on ACM SIGOPS European workshop*. 2002, pp. 102–107.
- [31] Amril Syalim and Dewangga Putra Sheradhien. “C vs Rust: Manual vs Automatic Spatial and Temporal Memory Safety”. In: *The Indonesian Journal of Computer Science* 14.2 (2025).
- [32] Lucian Radu Teodorescu. *Safety, Revisited*. URL: <https://www.accu.org/journals/overload/32/179/teodorescu/>.

- [33] Gábor Tóthvári, Kristóf Umann, and Zoltán Porkoláb. “Finding bugs on tagged unions mismatches”. In: *2024 IEEE 17th International Scientific Conference on Informatics (Informatics)*. IEEE. 2024, pp. 415–420.
- [34] Wm A Wulf and Sally A McKee. “Hitting the memory wall: Implications of the obvious”. In: *ACM SIGARCH computer architecture news* 23.1 (1995), pp. 20–24.
- [35] Wei Zhao et al. “CapSan-UB: Bug Capabilities Detector Based on Undefined Behavior”. In: *Proceedings of the 2023 13th International Conference on Communication and Network Security*. 2023, pp. 6–11.